# 分布式机器学习框架Ray介绍

目录

正文 2289 字（3 页） | 注释 101 字（2 个） | 浏览 8 次（1 人） | 全部 >

AI 速览 点击生成简要速览，支持更多 风格切换 助你高效阅读

## 目录

1 基本概念

2 分布式并行原理拆解

2.1 计算图

2.2 架构图

3 代码示例

3.1 Ray Task

3.2 Ray Actor

3.3 Ray Parameter Server

4 verl中的Ray

5 基于Ray的开源项目示例

---

仓库链接： https://github.com/ray-project/ray

官方文档： https://docs.ray.io/en/latest/index.html

原文链接： Ray: A Distributed Framework for Emerging AI Applications （2018年，一作为UCB的 Philipp Moritz ，也是TRPO和GAE论文的合作者，S. Levine课题组）

一句概括：Ray是一个**分布式**机器学习计算框架，以**装饰器**的形式出现在代码里(@ray.remote)，尤其适配**多节点**多卡模式，最初为了方便DRL开发部署。底层架构的核心技术是**多节点计算调度**和**分布式对象存储**。Ray在verl框架下主要用于分布式资源管理。

## 1 基本概念

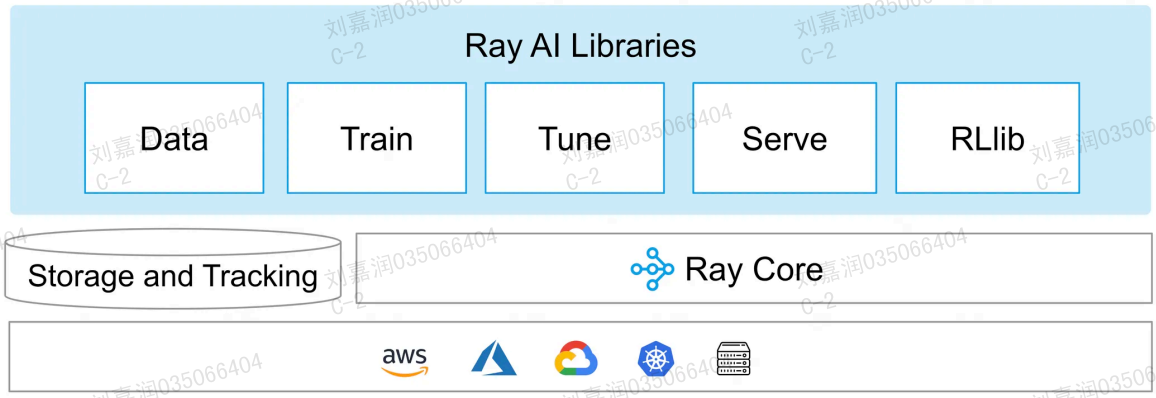

图1 Ray应用架构

Ray是一种分布式机器学习架构，最初目的是让开发人员能将运行在本地的原型算法，通过**少量代码添加**，即可快速转换成高效的**分布式应用**程序。该应用程序可以高效地在计算集群或者多核服务器上运行，而且代码的额外行数较少。Ray可以理解为专门针对AI算法（尤其是DRL算法）设计的分布式运行框架，与PyTorch、Tensorflow等框架兼容，支持CPU/GPU资源的自动通信和调度，同时支持DRL的训练+推理+仿真的全流程。

如图1所示，整个Ray的应用架构可以分为Ray AI Libraries和Ray Core两个部分，其中Ray AI Libraries可以理解为封装好的API生态库，Ray Core是灵活度较高的底层接口。我们主要关注Ray Core部分，也就是底层分布式计算逻辑这块。Ray Core有三个核心概念：

1）任务（Tasks）：在集群中异步执行的无状态函数。代码层就是一个**函数**。在task执行期间会占用一个worker，执行结束后会立刻释放这个worker的计算资源。

2）执行器（Actors）：集群中创建的有状态的运行进程。物理层类似就是某个cpu的进程worker，一般执行器常用作参数服务器[1]；代码层是一个**类**。actor会创建并一直占用一个worker process，直到整个ray被shut down才会被释放。

3）对象（Objects）：所有集群可以访问的不变的值。例如就是一个函数的return**值**。Object有两种创建的方法，一种是通过Task返回，每个Task的返回结果都会存储在本地的对象存储当中；另一种是使用ray.put()显式地创建，可以通过ObjectRef来引用Object，ObjectRef是Task和ray.put()的返回值。如果有Task或者ray.get()依赖于未完成创建的ObjectRef，会等待其创建完成再执行。

在Python的一般多进程程序中，子进程都是无状态的，如果想要给子进程传递一个大的参数，主进程就需要使用Pickle给每个进程都传递一份参数，这在多个进程维护同一个参数时效率很低。Ray做到将这些参数抽象化为对象存储在共享内存里，这样只需要给子进程传递一个“指针”[2]指向内存里的对象，子进程就能快速访问该参数而不需要反复在进程中传递数据。

# 2 分布式并行原理拆解

## 2.1 计算图

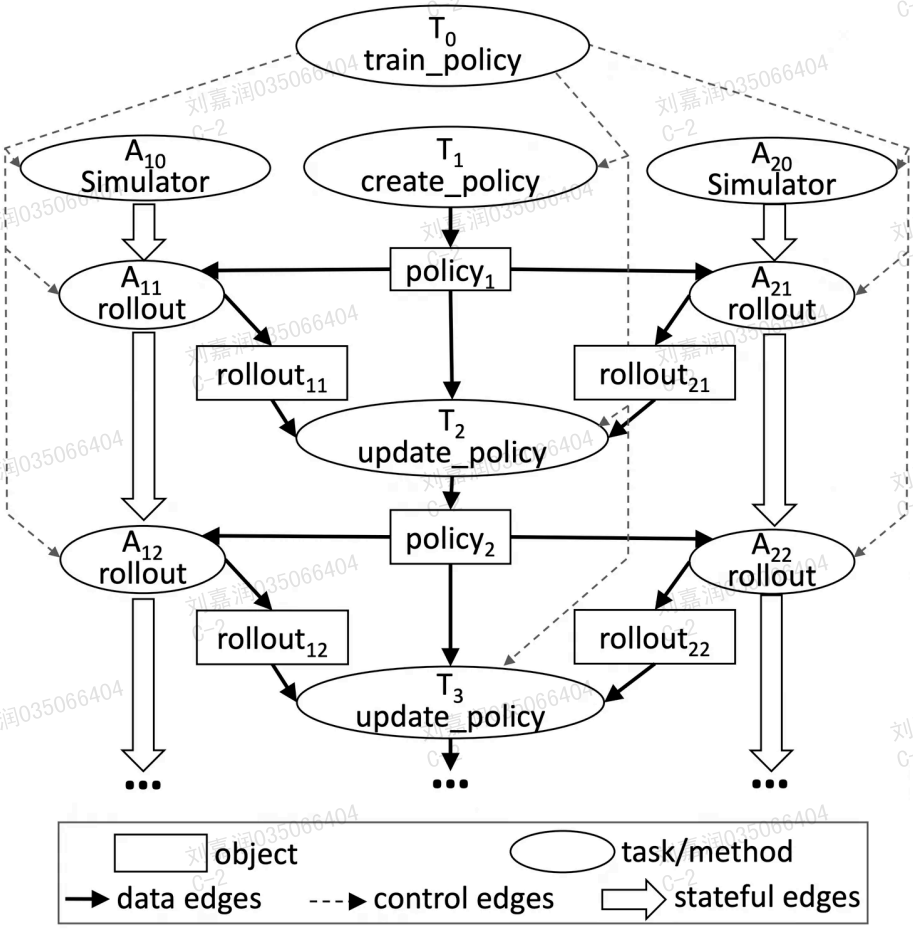


图2 Ray动态任务执行的状态转移图

Ray的计算flow基于动态任务图（与PyTorch类似，区别于Tensorflow的静态图）。如图2所示为Ray架构下两个环境采样的数据更新同一个策略模型的状态转移图。对象可以基于任务产生，并会阻塞下一待执行任务，如图2中的黑色实线箭头所示，只有当所需的对象全部准备好后，下一任务才会执行。

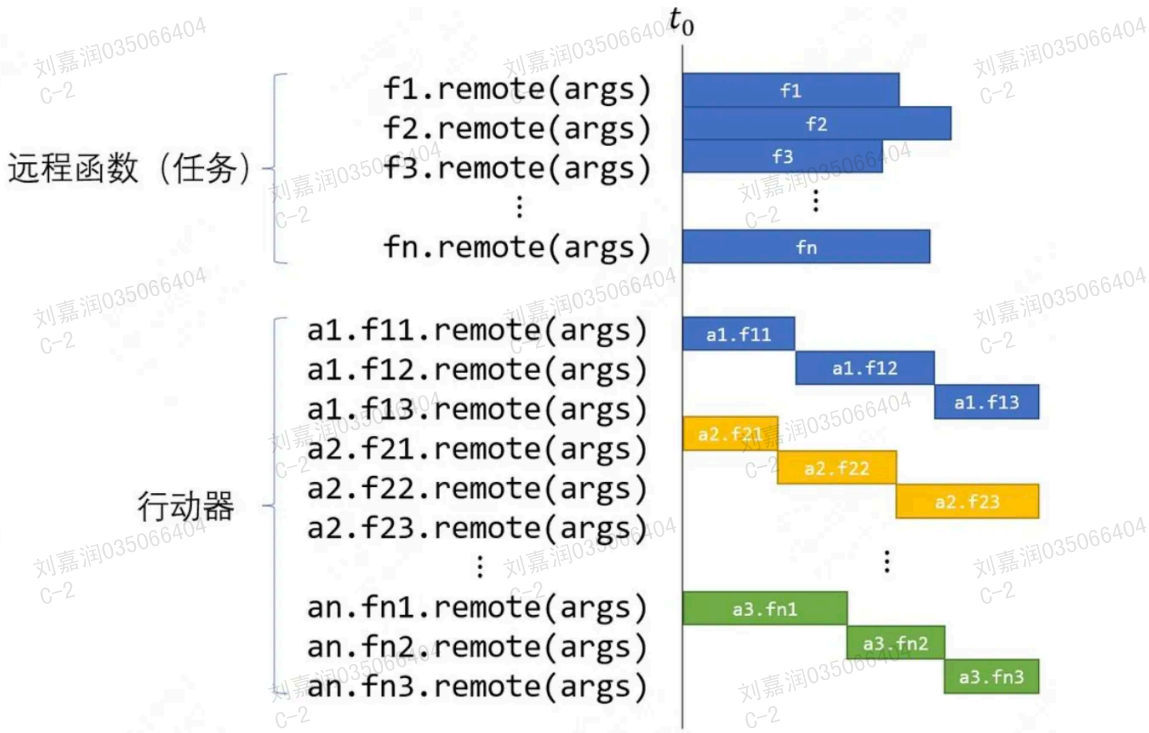


图3 任务和执行器的执行区别

如图3所示，对于任务和执行器的区别，在代码中定义的任务是并行执行的，但是在同一个执行器下的任务是串行执行，不同执行器之间互相并行。

## 2.2 架构图

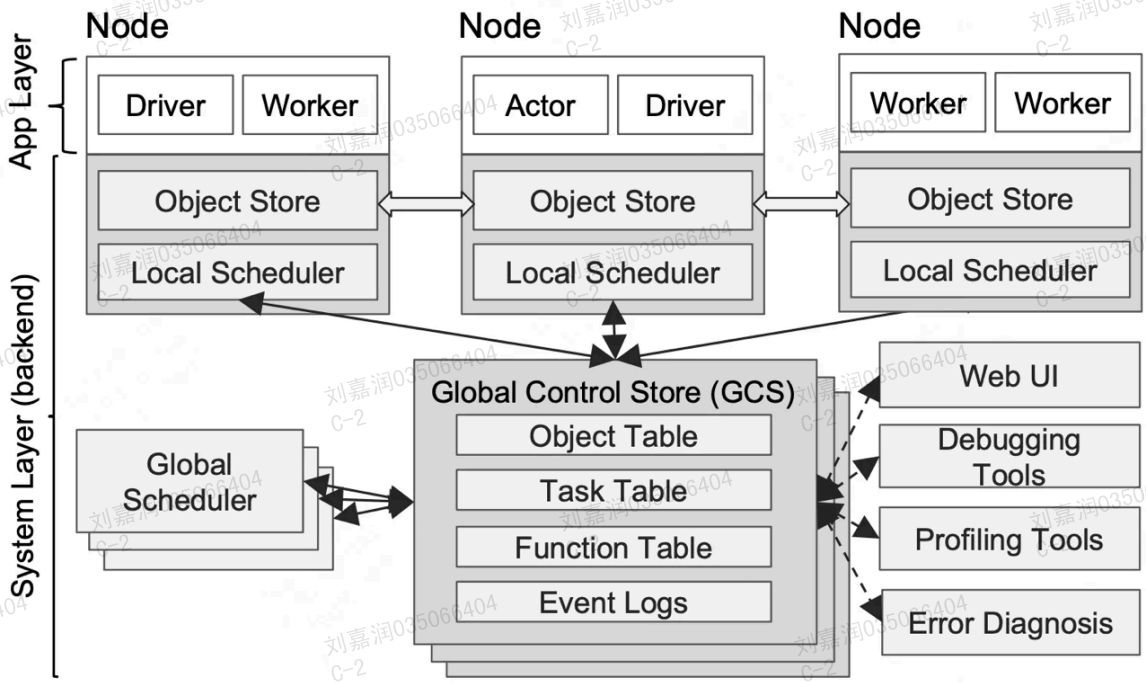


图4 系统架构图（先全局调度，再局部调度，事件驱动触发）

Ray架构相比于其他分布式架构最大的特点在于更适配多节点分布式计算的情况，不只限于单机多卡的分布式运行。如图4所示为基于Ray架构部署多节点训练的示例图，整个过程依赖两个调度器完成，分别是全局的global scheduler以及每一个节点的local scheduler。

当用户在Ray上提交一个任务时，这个任务首先会被发送到一个全局调度器。基于全局信息，全局调度器会首先做出决策，选择最合适的节点来执行这个任务。一旦任务被分配到了某个节点，该节点上的本地调度器就会接管，负责具体的任务执行。本地调度器会管理单节点内的资源，确保任务能够高效执行。执行过程中产生的数据，会被存储在分布式的对象存储系统（类似HDFS）中，方便其他节点的快速访问。

# 3 代码示例

## 3.1 Ray Task

Ray Task 代码块 Python

```python
import ray
ray.init()

@ray.remote
def f(x):
    return x * x

futures = [f.remote(i) for i in range(4)]
print(ray.get(futures)) # [0, 1, 4, 9]
ray.shutdown()
```

## 3.2 Ray Actor

Ray Actor 代码块 Python

```python
import ray
ray.init()

@ray.remote
class Counter(object):
    def __init__(self):
        self.n = 0

    def increment(self):
        self.n += 1

    def read(self):
        return self.n

counters = [Counter.remote() for i in range(4)]
[c.increment.remote() for c in counters]
```

```python
futures = [c.read.remote() for c in counters]
print(ray.get(futures)) # [1, 1, 1, 1]
ray.shutdown()
```

## 3.3 Ray Parameter Server

Ray Parameter Server 代码块 Python

```python
import ray
import numpy as np
ray.init()

@ray.remote
class ParameterServer:
    def __init__(self):
        self.params = np.zeros(16)

    def get_params(self):
        return self.params

    def update_params(self, gradients):  # 所有更新发生在参数服务器内部，用
死，防止更新冲突
        self.params += gradients

@ray.remote
class Worker:
    def __init__(self, ps):
        self.ps = ps

    def compute_gradients(self):
        return np.random.rand(16)

    def train(self):
        params = ray.get(self.ps.get_params.remote())
        gradients = self.compute_gradients()
        self.ps.update_params.remote(gradients)
        return np.sum(gradients)

ps = ParameterServer.remote()
start_params = ray.get(ps.get_params.remote())
print("初始参数: ", start_params)

# 创建4个worker
```

```
num_workers = 4
workers = [Worker.remote(ps) for _ in range(num_workers)]

# 模拟训练
num_iterations = 10
all_gradient_sum = 0
for i in range(num_iterations):
    results = ray.get([worker.train.remote() for worker in workers])
    all_gradient_sum += sum(results)
    print(f"第{i+1}轮, 每个worker梯度和分别为: {results}")

final_params = ray.get(ps.get_params.remote())
print("最后参数:", final_params)
print("全部worker累计梯度和: ", all_gradient_sum, "参数服务器更新的全部梯度和
sum(final_params)) # >> 二者相等
ray.shutdown()
```

## 4 verl中的Ray

verl里对Ray的定位是支撑分布式训练的资源调度，并对Ray进行了重新封装（verl/single_controller）。底层按照**Ray driver负责数据流、Ray worker负责计算的逻辑，以解耦数据流（原文也叫控制流）和计算流。**

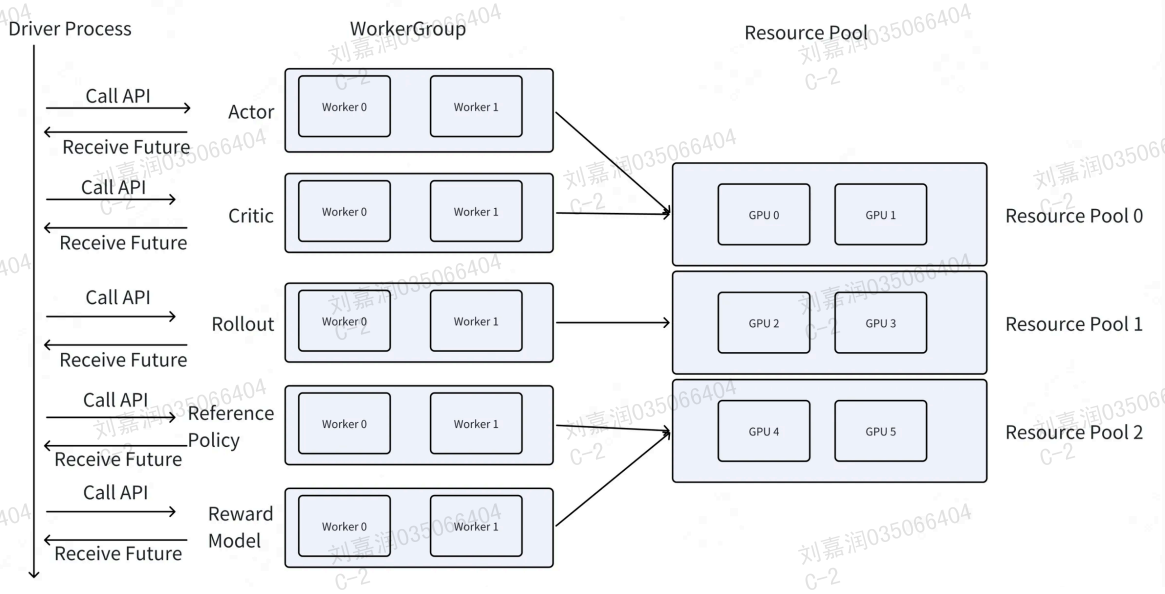


图5 verl执行示意图

如图5是verl强化学习作业执行的示意图。Driver在单个进程上运行（Single Controller），而Generator/Actor/Worker等则在多个进程上运行，并分别放置在特定的资源组中调用对应的计算资源。在Rollout阶段，Driver将数据传递给生成器执行Rollout。Rollout完成后，数据将传回控制器，用于下一步计算。整个执行过程采用混合控制器设计，数据流和计算分离解耦，以提高计算效率、增强定义算法训练循环的灵活性。数据并行要求的通信（比如做梯度同步、参数共享等）Ray并不支持，这个时候还是要使用torch.distributed来完成。

verl中与Ray相关的几个核心概念：

1）Worker：一个无状态的计算线程，受到Driver的控制。多个verl worker可能只对应一个ray actor，因此多个verl Worker可以共享资源。verl里面的Worker和Ray的Worker含义不一样。

2）WorkerGroup：生成一个WorkerGroup的instance时，需要指定ResourcePool和ClassWithInitArgs。一个WorkerGroup instance运行在driver process里面。同时会在ray cluster中启动一组ray worker process，并在每个ray worker process中运行一个ray actor - 具体启动多少actor以及怎样分配资源，由ResourcePool指定 - 启动什么样的ray actor，由ClassWithInitArgs指定。在ClassWithInitArgs中会指定一个类用来创建worker。 每个workgroup有一个master worker。

3）ResourcePool：资源池，用于定义WorkGroup中的进程调用哪些GPU资源。

4）DriverProcess：主控进程，主要用于控制数据流和计算流。

## 5 基于Ray的开源项目示例

Ray RLlib ：可扩展的强化学习库，可在许多机器上运行，支持A3C/DQN/PPO等。

Ray Tune ：高效的分布式超参数搜索库。它提供了用于深度学习，强化学习和其他计算密集型任务的Python API。

verl ：字节火山引擎开发的基于RL的LLM后训练/微调库，底层调用Ray接口。

注释：

[1] 参数服务器（Parameter Server）是一种分布式机器学习系统架构，主要用于大规模机器学习模型的训练和部署。它的核心思想是将模型参数存储和更新的任务从梯度计算和推理任务中分离出来，以实现更高效的并行计算和更好的可扩展性

[2] 这个“指针”称为ObjectRef

仅供内部使用，未经授权，切勿外传